Machine Learning Project Report

Car Price Prediction

---

1. Introduction

This project focuses on predicting the selling price of used cars using machine learning techniques. The dataset contains various features such as car name, year, fuel type, transmission, mileage, engine capacity, and more. Two models are used for prediction:

- Linear Regression
- Random Forest Regressor

---

2. Libraries Used

The following Python libraries are used in this project:

- pandas (data manipulation)
- numpy (numerical operations)
- seaborn (data visualization)
- matplotlib (plotting)
- sklearn (machine learning models and preprocessing)

---

3. Data Loading

The dataset is loaded using pandas:

```
data = pd.read_csv('Cardetails.csv')
```

---

4. Exploratory Data Analysis (EDA)

Basic exploration is performed to understand the dataset:

- Preview data:

```
data.head()
```

- Check shape:

```
data.shape
```

- Data types:

```
data.info()
```

- Missing values:

```
data.isnull().sum()
```

- Unique values per column:

```
for col in data.columns:
    print(col, ":", data[col].nunique())
```

---

5. Data Preprocessing

5.1 Removing Unnecessary Data

- Dropped irrelevant column:

```
data.drop(columns=['torque'], inplace=True, errors='ignore')
```

5.2 Handling Missing Values

```
data.dropna(inplace=True)
```

5.3 Removing Duplicates

```
data.drop_duplicates(inplace=True)
```

---

6. Encoding Categorical Variables

Label Encoding is applied to convert categorical data into numerical form:

- Car name
- Fuel type
- Seller type
- Transmission
- Owner type

Example:

```
name_le = LabelEncoder()
data['name'] = name_le.fit_transform(data['name'])
```

---

7. Feature Engineering

7.1 Extract Numerical Values from Text Columns

- Mileage:

```
data['mileage'] = data['mileage'].str.extract(r'(\d+\.?\d*)').astype(float)
```

- Engine:

```
data['engine'] = data['engine'].str.extract(r'(\d+)').astype(float)
```

- Max Power:

```
data['max_power'].str.extract(r'(\d+\.?\d*)').astype(float)
```

7.2 Filling Missing Values

Missing values are replaced with mean:

```
data['mileage'].fillna(data['mileage'].mean(), inplace=True)
```

---

8. Feature Scaling

Standardization is applied to numerical columns:

```
scaler = StandardScaler()
numerical_cols = ['year', 'km_driven', 'mileage', 'engine', 'max_power', 'seats']
data[numerical_cols] = scaler.fit_transform(data[numerical_cols])
```

---

9. Data Visualization

Correlation heatmap is used to analyze relationships:

```
sns.heatmap(data.corr(), annot=True)
plt.pyplot.show()
```

---

10. Train-Test Split

Dataset is divided into training and testing sets:

```
X = data.drop("selling_price", axis=1)
Y = data["selling_price"]
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)
```

---

11. Model Training

11.1 Linear Regression

```
model = LinearRegression()
model.fit(X_train, Y_train)
```

11.2 Random Forest Regressor

```
rf_model = RandomForestRegressor(random_state=42)
rf_model.fit(X_train, Y_train)
```

---

12. Model Evaluation

Linear Regression Scores:

```
model.score(X_test, Y_test)
model.score(X_train, Y_train)
```

Random Forest Scores:

```
rf_model.score(X_test, Y_test)
rf_model.score(X_train, Y_train)
```

---

13. Prediction

13.1 Prediction using Test Data

```
rf_model.predict([[1314,0.869379,-0.988515,1,1,1,0,1.194090,-0.370633,0.023135,-0.441650]])
```

13.2 Prediction using New Input Data

```
new_data = pd.DataFrame([["Skoda Rapid 1.5 TDI
Ambition",2014,120000,"Diesel","Individual","Manual","Second Owner","21.14 kmpl","1498
CC","103.52 bhp",5]], columns=X_train.columns)
```

Steps:

- Encode categorical values
- Extract numerical values
- Apply scaling
- Predict using model

```
rf_model.predict(new_data)
```

---

14. Conclusion

- Random Forest Regressor generally performs better than Linear Regression.
- Proper preprocessing (encoding, scaling, cleaning) significantly improves model performance.
- The model can be used to estimate car prices based on given features.